project-proposal-formatter 用户手册

给**中国式项目申报类文档**（项目建议书 / 申报书 / 任务书 / 投标书）套用特定机构排版规范的 Skill。

规范按「一类型一文件」组织在 `references/` 下。目前收录：

机构	文档类型	规范文件	版本
国防科工局（SASTIND）	项目**建议书**	`references/format-proposal-sastind.md`	V4.4
国防科工局（SASTIND）	项目**任务书**	`references/format-taskbook-sastind.md`	V1.4
工业和信息化部（MIIT）	项目**技术标书**	`references/format-bid-miit.md`	V4

> MIIT 技术标书另配一份**可直接使用的 Word 样式模板**：`assets/miit-techbid-template.docx`。

这个 Skill 能做什么

输入一份排版混乱的草稿（`.docx`），输出一份套好规范的成品。处理范围包括：

- **字体字号**：正文中文字体 + 西文/数字字体分层设置（`w:eastAsia` 与 `ascii/hAnsi` 分开，避免数字外观被改）
- **段落**：行距、首行缩进 2 字符、两端对齐、**关闭「与文档网格对齐」**（SASTIND § 1.1——否则行距会被文档网格吸附，设了却不生效）
- **标题**：各级标题样式统一，并清理 run 级直接格式（否则会覆盖样式定义）
- **章节分页**：按文档类型分别处理——建议书 = 各一级章节从新页开始且不重复分页；任务书 = **取消**一级标题分页（`w:pageBreakBefore` 一律移除），但保留承担页码体系的分节符
- **图表题注**：统一为 Word `Caption` 样式，识别并把手打的纯文本题注转成真正的 SEQ 域，编号顺延纠错
- **表格**：居中 + 表内字号统一 + 表头加粗（MIIT）
- **目录**：SASTIND = 主目录 + 图目录 + 表目录；MIIT = 最多 5 级自动目录，独立起页于正文首个一级标题之前
- **页码**：SASTIND = 前置区罗马数字、正文阿拉伯数字重新编号；MIIT = 单节连续阿拉伯数字、封面页即第 1 页
- **异常处理**：章节漏号（Word 自动编号造成的假阳性）、异常字符

⚠ 选错规范文件会出大事

同一机构也可能有多套规范，先定「文档类型」再定「机构」。 拿错文件不只是字号错，还可能把行为做反。

两套 SASTIND 规范的区别：

	项目建议书	项目任务书
标题体系	**三级**，一级**四号不加粗**、二三级小四加粗	**四级**，一～四级**统一宋体小四 + 加粗**
正文行距	1.5 倍	**1.25 倍（表格除外、各级标题除外**，两者仍 1.5）
标题样式名	「建议书一/二/三级标题」	「任务书一/二/三/四级标题」

	项目建议书	项目任务书
一级标题分页	**强制分页**（章节各起新页）	**不强制分页**，并清除已有的 `w:pageBreakBefore`
与文档网格对齐	全文取消（§ 1.1）	全文取消（§ 1.1）· **两文档一致**
其余	—	与建议书规范逐段一致

> 任务书规范是建议书规范 V4.3 的**复制派生**（两份都是自包含全文，可独立执行）。差异共 9 项（8 项外观 + 1 项分页行为），列在任务书文件开头的速查表里（第 10 项「网格对齐」登记为"同"）；**改了一份要评估是否同步另一份**——底本的通用机制（题注、分节、页码、网格对齐、校验法）应同步，**建议书专属口径（如强制分页）绝不能同步**。
>
> ⚠ **「一级标题分页」是唯一"改行为"的差异**，脚本层面也是反的：建议书 STEP 5 是"缺分页就补"，任务书 STEP 4 是"有分页就删"，**不可互换**。

MIIT 技术标书规范与 SASTIND 的差异（两个体系完全不同，别混用）：

	SASTIND（建议书/任务书）	MIIT（技术标书）
中文字体	宋体	**仿宋**
一级标题	新建自定义样式「建议书/任务书一级标题」	**重定义 Word 内置「标题 1~5」**
标题编号	汉字序号（一、二、三）**手打**	阿拉伯多级编号 `1 / 1.1 / 1.1.1`，**手打**
标题缩进	首行缩进 2 字符	**悬挂缩进 1.27 cm（五级统一）**
正文行距	1.5 倍（任务书 1.25 倍）	**固定值 28 磅**
页码	前置罗马 / 正文阿拉伯	**单节连续，封面即第 1 页**
封面	一并纳入前置内容处理	**完全不动**
目录	主目录 + 图目录 + 表目录	**最多 5 级、独立起页于正文首个一级标题之前**

目录结构

```
project-proposal-formatter/
├── SKILL.md                                  # 技能入口：触发条件、工作流、扩展方式
├── README.md                                 # 本文件
├── assets/
│   └── miit-techbid-template.docx            # MIIT 技术标书 Word 样式模板（含自动目录页）
├── references/
│   ├── format-proposal-sastind.md            # 国防科工局 项目建议书 格式要求（V4.4）
│   ├── format-taskbook-sastind.md            # 国防科工局 项目任务书 格式要求（V1.4）
│   └── format-bid-miit.md                    # 工信部 项目技术标书 格式标准（V4）
└── scripts/
    ├── diagnose_docx.py                      # 离线诊断：规模/样式/漏号/异常字符/网
```

```
格对齐
    ├── format_sastind_proposal.py          # 建议书格式调整参考实现（python-docx
离线路径）
    ├── format_sastind_taskbook.py          # 任务书格式调整参考实现（python-docx
离线路径）
    ├── build_miit_template.py              # 生成 MIIT Word 样式模板
    ├── check_miit_heading_numbers.py       # 手打标题编号纠错（--fix 可自动修正）
    └── verify_miit_toc.py                  # 目录页校验（Word 往返 + PDF 坐标核
对）
```

安装

把整个 `project-proposal-formatter/` 目录放进你的 Skill 目录：

- **用户级**（所有项目可用）：`~/.workbuddy/skills/`
- **项目级**（团队共享）：`<项目根>/.workbuddy/skills/`

```
git clone <本仓库地址> ~/.workbuddy/skills/project-proposal-formatter
```

或直接解压 `dist/project-proposal-formatter.zip`（zip 内第一层就是 `project-proposal-formatter/`，解压到上面的目录即可，不会多套一层）。

懒人安装方式：直接告诉workbuddy,说帮我按照project-proposal-formatter技能，下载地址为：https://github.com/123wszk/project-proposal-formatter.git

依赖

- **Python 3** + `python-docx`（离线脚本路径用）

  ```
  pip install python-docx lxml
  ```

- **可选**（仅 MIIT 相关工具需要）：`pywin32`（Word COM 校验）、`pypdfium2`（PDF 栅格化核对）
- **文档编辑通道**（可选，取决于宿主环境）：本 Skill 只负责「格式规范 + 编排流程」，实际文档编辑可交给宿主的本地文档编辑能力（如实时编辑器 MCP），也可直接用离线脚本。若你的环境没有这些，走 `scripts/` 的离线路径。

快速开始

```
# 1. 先诊断文档现状（建议书、任务书通用；MIIT 也适用）
python scripts/diagnose_docx.py 你的文档.docx

# 2a. 套用「建议书」规范
python scripts/format_sastind_proposal.py 你的建议书.docx 输出.docx

# 2b. 套用「任务书」规范
python scripts/format_sastind_taskbook.py 你的任务书.docx 输出.docx
```

```
# 2c. 生成「MIIT 技术标书」空白模板（新建标书时用）
python scripts/build_miit_template.py miit-techbid-template.docx

# 3. MIIT 配套校验
python scripts/check_miit_heading_numbers.py 你的标书.docx      # 手打编号纠错
python scripts/verify_miit_toc.py 你的标书.docx                  # 目录页校验

# 调试：分步存档以二分定位问题（STAGES=1）
STAGES=1 python scripts/format_sastind_taskbook.py 输入.docx 输出.docx
```

> ⚠ **两个 SASTIND 脚本都是「参考实现 / 起手模板」，不是开箱即用的通用工具**，都含稿件特定硬编码（样式名、编号与题注策略）。移植到新稿前，**先读脚本 docstring 顶部的「必须按稿调整」清单**，并用 `diagnose_docx.py` 核对实际样式名与结构。
>
> - `format_sastind_proposal.py` 对应**建议书**规范：三级标题、一级标题宋体四号不加粗、全文 1.5 倍行距。
> - `format_sastind_taskbook.py` 对应**任务书**规范：四级标题（新建 4 个样式）、一～四级统一宋体小四加粗、正文区 1.25 倍行距（表格·前置·题注保持 1.5）、**一级标题"清分页"（STEP 4）**、题注连续编号 + SEQ 域、主目录过期缓存清理。两个脚本都会在"全文打底行距"那一轮**同时关闭网格对齐**（`w:snapToGrid w:val="0"`）。
> - **别拿建议书脚本跑任务书**（或反之）——不仅字号体系会整体错位，**分页逻辑还会做反**：建议书脚本会主动给一级标题**补**分页，而任务书要的是**清**分页。

处理新机构 / 新文档类型

1. 在 `references/` 新建 `format-<文档类型>-<机构简称>.md`（**文档类型在前**，如 `format-application-nsfc.md`）。
2. 套用 `SKILL.md` 里的 reference frontmatter 模板，节数按实际要求增减，不要为凑模板硬写空节。
3. 转写用户给的格式要求文档时**逐段逐条核对**，并主动补全原文没写但实操必备的维度（前置内容、页码体系、章节分页、已有目录/题注处理、异常字符、工具能力边界）。
4. 把「原文没写、由推断补的」条目单独列为**「待用户确认」清单**放在文件末尾。

同一机构再加一种文档类型时，请用 `SKILL.md` 里的**「复制派生法」**：先 `cp` 旧规范保底（技术条款一字不丢），再在开头加「差异速查表 + 版本记录 + 底本沿革」，然后逐项外科式改写（正文条款 + § 13 流程 + 验收清单）。

详见 `SKILL.md` 的 *Extending the Skill* 一节。

已知限制

- **域（Field）刷新**：目录、题注编号、交叉引用都是 Word 域。`.docx` 保存时缓存结果不会自动更新，交付后需在 Word/WPS 中按 `Ctrl+A` → `F9` 刷新一次。（MIIT 模板已置 `settings.xml/updateFields=true`，打开时会提示更新。）
- **无 Word COM 的环境**：Linux + WPS 等环境下无法通过自动化接口刷新域，只能靠人工按 F9；同时 WPS 命令行无头转 PDF 在部分发行版上静默失败，无法据此自校验。此时改用 references § 11 的 6 项结构性校验（重开 package / `unzip -t` / lxml 良构性 / rels 断链 / OOXML 子元素顺序 / 逐节点 diff）。
- **可选项默认跳过**：如「错别字自检」属可选项，默认不执行，需用户明确要求才跑。

- **行距"设了没生效"（SASTIND）**：若文档在 `w:sectPr/w:docGrid` 里定义了行网格，段落还勾着「如果定义了文档网格，则与网格对齐」时，行距会被吸附到网格行的整数倍（1.25 倍与 1.5 倍可能落到同一档）。本 Skill 的规范（§ 1.1）要求**全文取消**该勾选，脚本已实现；手工操作时在 Word 里 `Ctrl+A` → 段落 → 取消勾选即可。
- **两份 SASTIND 规范是复制关系**：不是引用关系，一份有更新时需人工评估是否同步到另一份。

版本

- `format-proposal-sastind.md`：**V4.4**（V4.4 新增 § 1.1 网格对齐：全文取消「与文档网格对齐」，段落级 `w:snapToGrid w:val="0"`）
- `format-taskbook-sastind.md`：**V1.4**（派生自建议书 V4.3；V1.1 补入 3 条实测踩坑 + 配套任务书脚本；V1.2 取消一级标题强制分页并改为清除已有分页属性；V1.3 同步建议书 V4.4 的 § 1.1 网格对齐；V1.4 明确"1.25 只作用于正文文字段落、各级标题仍是 1.5"，并修掉"标题样式写 `w:line=300` / 段落实际 360"的规范-实现不一致）
- `format-bid-miit.md`：**V4**（V1 转写用户草案；V2 编号改手打 + 表头加粗；V3 一级悬挂统一 1.27cm + 西文 Times New Roman + 页码单节连续从封面起为 1 + 新增手打编号纠错；V4 新增最多 5 级、独立起页的目录页）
- `assets/miit-techbid-template.docx`：随 `format-bid-miit.md` V4 重出（3 页：封面位置 / 目录 / 正文示例）
- 变更记录见各文件顶部的元信息。

许可

未附加许可证文件。上传到 GitHub 前请按需补充 `LICENSE`（如 MIT / Apache-2.0）。

使用案例



使用建议

非涉密项目：建议采用workbuddy自带的基础大模型，相应速度较快

涉密项目：采用公司内部大模型，安全性较高，但相应速度慢

如何在workbuddy中配置使用公司内网的大模型?

一、登录“https://aiapi.phytium.com.cn”申请API KEY

二、依次点击“API编程接口”——“LiteLLM 管理面板 /ui”——“create New Key”

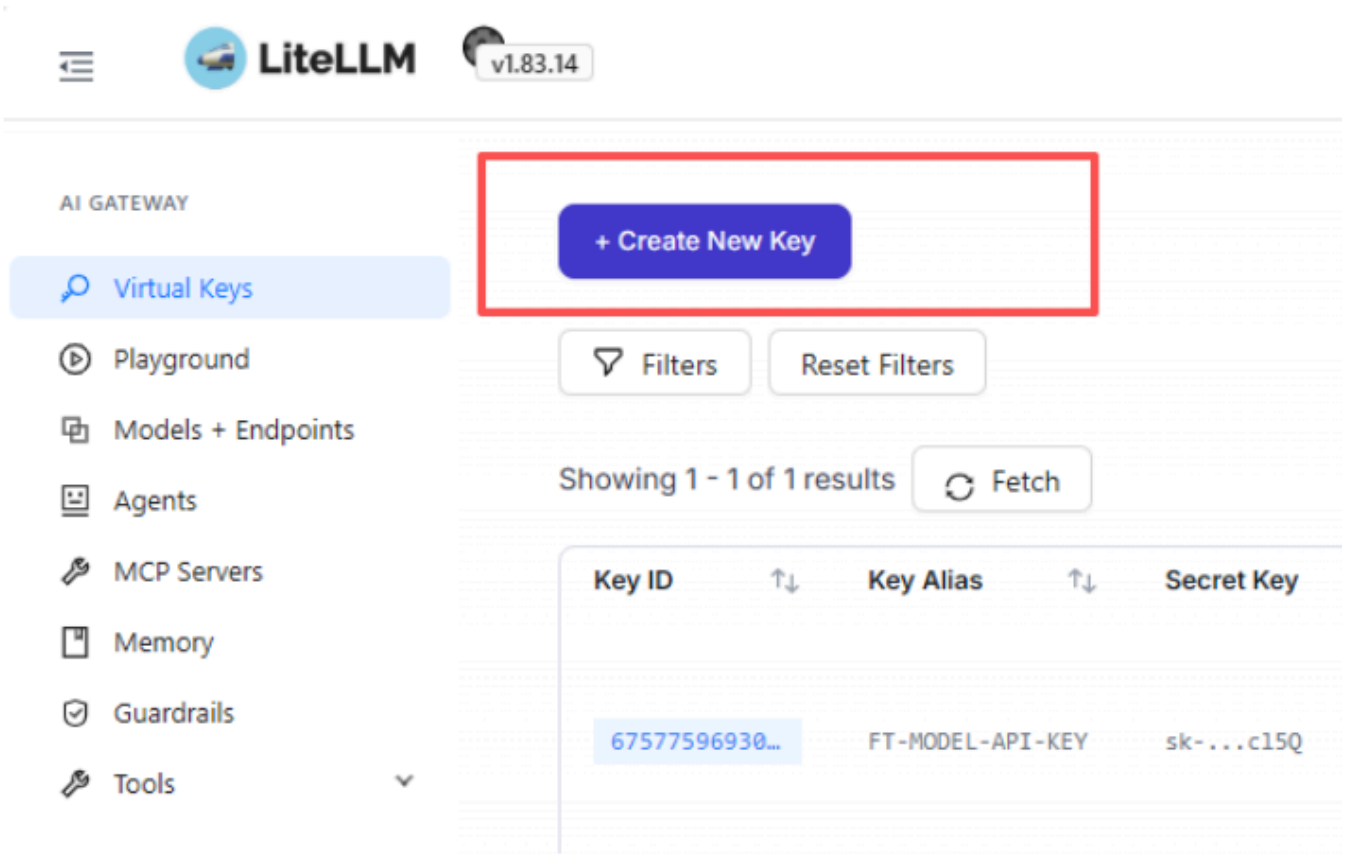
LiteLLM API 1.83.14 OAS 3.1

/openapi.json

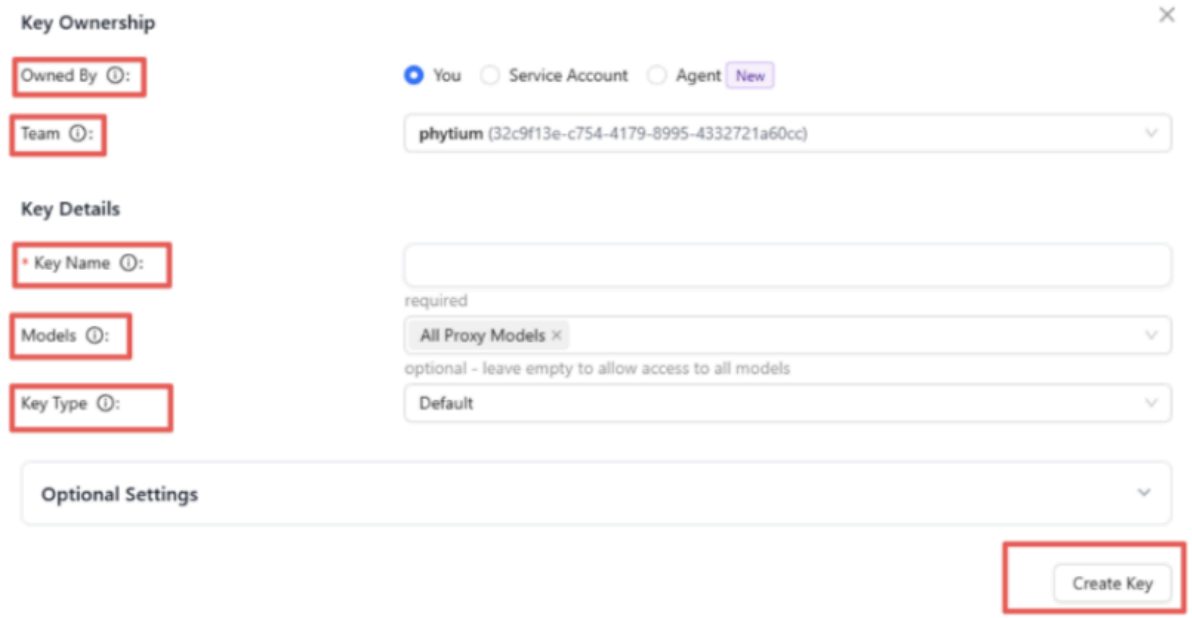
Proxy Server to call 100+ LLMs in the OpenAI format. Customize Swagger Docs

👉 LiteLLM 管理面板 /ui . 使用 SSO 创建、编辑密钥

🔑 LiteLLM 模型中心 . 查看代理上可用的模型。文档



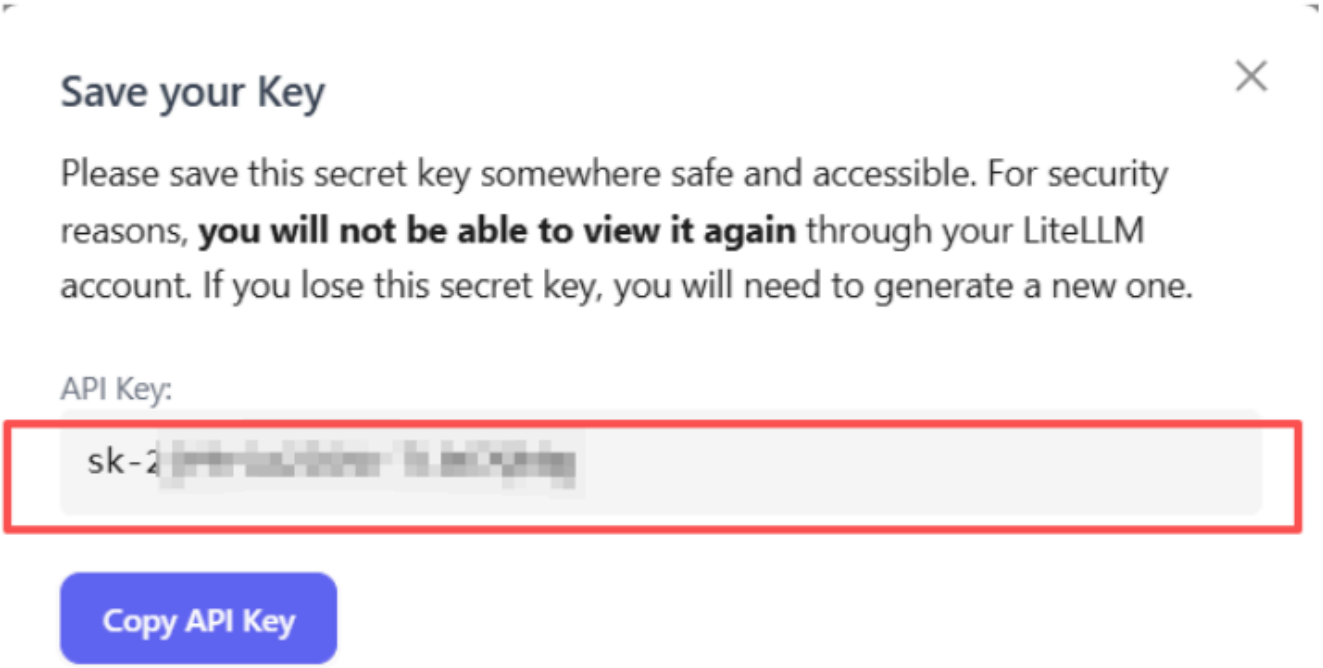
详细的API KEY设置，参考连接：https://aiclub.phytium.com.cn/pages/viewpage.action?pageId=148242523



配置参数说明：

参数	说明	建议
Owned By	选择所有者	选择"You"
Team	选择团队	选择默认的"phytium"
Key Name	密钥名称	**全局唯一**，避免冲突
Models	可用模型	根据需要选择，可多选
Key Type	密钥类型	务必选择**Default**

> **重要提醒：配置完成后，点击Create Key按钮，此时会弹出对话框，切记保存对话框中的Key，此对话框是唯一且仅且一次显示。**



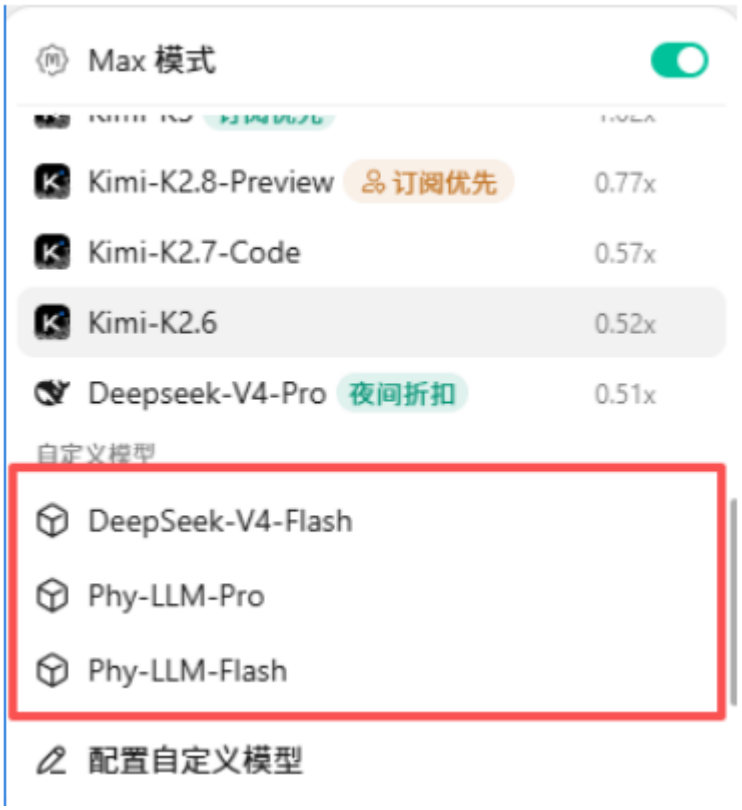
三、得到API KEY后，点击workbuddy中的"配置自定义模型"，FT内网大模型API接口地址为：
https://aiapi.phytium.com.cn/v1/chat/completions

配置细节如下：





四、配置完成后，可在自定义模型中查看配置结果并开始使用


Max 模式
Kimi-K2.8-Preview
订阅优先
0.77x
Kimi-K2.7-Code
0.57x
Kimi-K2.6
0.52x
Deepseek-V4-Pro
夜间折扣
0.51x
自定义模型
DeepSeek-V4-Flash
Phy-LLM-Pro
Phy-LLM-Flash
配置自定义模型